

NATIONAL UNIVERSITY OF SINGAPORE
SCHOOL OF COMPUTING
MIDTERM TEST
AY2024/25 Semester 1
CS2100 – COMPUTER ORGANIZATION
DURATION OF ASSESSMENT: 1.5 HOURS

- DRAFT ANSWERS -

We are providing the answers for this midterm to further your understanding of the subject. We hope you carefully review it and catch the part where you had an issue, improving your knowledge of the subject matter.

Part A: Multiple Choice Questions (25 marks)

Each MCQ has only one correct answer. Shade the corresponding circle on the Answer Sheets for your answer using a pencil, ensuring that the shading is prominent and the other options are not tainted. No other form of marking out your answer will be considered. We are using machine recognition to grade the MCQs.

Question 1

What is the result of the following subtraction in 8-bit signed magnitude representation:

$$00110010_{\text{sm}} - 10111101_{\text{sm}}$$

- A. 10010000
- B. 10010001
- C. **01101111**
- D. 11101110
- E. None of the above

Answer:

Since this is signed magnitude, it is equivalent to 7-bit (after taking away the sign bit) $0110010_2 + 0111101_2 = 110111_2$, option C.

Question 2

Bob decided to design a new processor that works with 137 bits integers. He decided also to use two's complement to represent signed integer. Which of the following in hexadecimal is the largest *positive* integer that can exist in his representation system?

- A. **0x0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF**
- B. 0x1FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
- C. 0x3FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
- D. 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
- E. None of the above

Answer:

Taking away one bit for the sign bit, we are left with 136 bits. This gives nicely 34 hexadecimal 'F's – which will be the largest positive integer, option A.

Question 3

This is related to Assignment 1 Question 2. Consider the following:

$$3a79_X - 2035_Y = 1792_Z$$

Which one of the following is **not** a valid solution?

- A. $X = 12, Y = 13$ and $Z = 11$
- B. $X = 15, Y = 16$ and $Z = 14$
- C. $X = 11, Y = 12$ and $Z = 10$
- D. $X = 13, Y = 15$ and $Z = 11$**
- E. None of the above

Answer:

There is no easy way around this one. You have to try and eliminate each.

Checking option A:

$$3a79_{12} = 6717_{10}$$

$$2035_{13} = 4438_{10}$$

$$6717 - 4438 = 2279$$

$$1792_{11} = 2279_{10}$$

So this turns out to be correct (the answer wants the incorrect one)

Checking option B:

$$3a79_{15} = 12489_{10}$$

$$2035_{16} = 8245_{10}$$

$$12489 - 8245 = 4244$$

$$1792_{14} = 4244_{10}$$

So this is also correct (not what the answer)

Checking option C:

$$3a79_{11} = 5289_{10}$$

$$2035_{12} = 3497_{10}$$

$$5289 - 3497 = 1792$$

$$1792_{10} = 1792_{10}$$

So this is also correct (not what the answer)

Checking option D:

$$3a79_{13} = 8381_{10}$$

$$2035_{15} = 6800_{10}$$

$$8381 - 6800 = 1581$$

$$1792_{11} = 2279_{10}$$

So this is the answer we want, coz the result of the subtraction does not match up.

Question 4

Which of the following in hexadecimal would represent the smallest positive normalized floating point number representable in the IEEE Standard 754 single precision (32-bit) floating point?

- A. 0x00000001
- B. 0x00800000**
- C. 0x00800001
- D. 0x01000000
- E. None of the above

Answer:

In the IEEE floating point format, the number represented is a normalized number if the biased exponent is not zero or the largest integer of the given number of bits (both these “slots” are reserved for special (not normalized) numbers and values such as infinities – zero being a special number).

For the smallest positive number, the sign bit must be 0. Given the above, the smallest biased exponent is 1. And the mantissa bit is all zero (with the implicit 1 “holding the fort”.) This gives us the answer being option B.

Question 5

What value does the hexadecimal 0xBEA00000 represent in the IEEE Standard 754 single precision (32-bit) floating point?

- A. 0.12565
- B. -0.12565
- C. 0.5125
- D. -0.3125**
- E. None of the above

Answer:

Straightforward decoding.

0xBEA00000 = 1011 1110 1010 0000 0000 0000 0000 0000

So sign bit (green) is 1, hence a negative number.

The biased exponent (brown) is $01111101_2 = 125_{10}$. This gives us a true exponent of -2.

The mantisaa is $1.01_2 = 1 + 0.25 = 1.25$.

So the answer is $-(1.25 \times 2^{-2}) = -0.3125$. Option D.

Question 6

Which of the following is the smallest positive base-10 value that is representable as a 32-bit integer but cannot be represented precisely as in IEEE Standard 754 single precision (32-bit) floating point? In other words, when one converts the said value (let's call it A) into the IEEE Standard 754 single precision (32-bit) floating point representation (let's call it F), and then convert F back into an integer (let's call it B), then A will not be equal to B.

- A. 16777215₁₀
- B. 16777216₁₀
- C. 16777217₁₀**
- D. 16777218₁₀
- E. None of the above

Answer:

We have only 23 bits in the mantissa of the IEEE Standard 754 single precision (32-bit) floating point to represent an integer. We do have an additional implicit "1". In addition, the burden of the sign is taken care of by the sign bit. So if we have a number (in binary) that starts with a '1' followed by 23 '0's – and then a '1' (a 25 bit number), then even utilizing the implicit '1' at the MSB, we still need 24 bits – one more than what we have. Hence this integer will not be representable. This number is 0x1000001, which is 16777217₁₀, option C.

Do note that this does not mean every integer after it will not be representable exactly. For instance, 0x2000000, although larger than our answer, is exactly representable because after normalization and dropping the zeros to the right, we have a mantissa with less than 23 significant bits (and a larger exponent), and hence is representable both as an 32-bit integer and a IEEE Standard 754 32-bit floating point number exactly.

Question 7

Which of the following is a valid C initialization statement (i.e., no compilation warning or error, and code will run accordingly)?

- A. `int A[4] = {1, 2, 3, 4, 5};`
- B. `int A[4] = {1, 2, 3};`**
- C. `int A[4] = [1, 2, 3, 4];`
- D. `int A[4] = {1, , 3, 4};`
- E. None of the above

Answer:

Option A: longer than 4.

Option B: this is the correct answer. The remaining one will be filled with a zero.

Option C: square bracket used in the initializer.

Option D: missing element.

Question 8

Consider the following C program:

```
for (int i=0; i<10; i++) {  
    printf("%d ", ++i);  
}
```

What output would be printed out at the terminal after compiling and executing this C loop?

- A. 1 2 3 4 5
- B. 0 2 4 6 8
- C. 1 3 5 7 9
- D. 1 2 3 4 5 6 7 8 9
- E. None of the above

Answer:

There are two increments of i. What is printed is the pre-incremented i. So the answer is C.

Question 9

Consider the following C program:

```
#include <stdio.h>

int x = 1;

int main(int argc, char *argv[])
{
    int x = 2;
    for (int x=0; x<4; x++) {
        int x = 3;
        printf("%d ", x++);
    }
    printf("%d\n", x);
}
```

What output would be printed out at the terminal after compiling and executing this program?

- A. 3 3 3 3 2**
- B. 3 3 3 3 3
- C. 3 4 5 6 7
- D. 0 3 4 6 8
- E. None of the above

Answer:

The innermost printf() prints its local variant of x – which is always initialized to 3. So 4 “3”s will be printed, even though x is post-incremented in the printf() because each time round this loop, a new x is instantiated. The final printf() prints the variant of x that is local to main() and which scopes out the global x. Hence it prints “2”. So the answer is option A.

Question 10

Suppose a small modification is made to the C program from Question 9 (underlined below):

```
#include <stdio.h>

int x = 1;
int main(int argc, char *argv[])
{
    int x = 2;

    for (int x=0; x<4; x++) {
        static int x = 3;
        printf("%d ", x++);
    }
    printf("%d\n", x);
}
```

What output would be printed out at the terminal after compiling and executing this program?

- A. 3 3 3 3 4
- B. 3 4 5 6 7
- C. 0 1 2 3 4
- D. 3 4 5 6 2**
- E. None of the above

Answer:

Here, the big difference is in the innermost printf() prints its local variant of x is now declared as "static". This means that there shall be only one instance of it. This will result in this same instance being incremented each time round the loop. So the answer is option D.

Question 11

Consider the following C program:

```
#include <stdio.h>

int A[4] = {1, 2, 3, 4};

int main(int argc, char *argv[])
{
    int x, y;
    int *p[2], **q;

    x = A[0];
    y = A[1];

    p[0] = &(A[1]);
    p[1] = &(A[3]);
    q = p;

    *q++ = &(A[2]);
    **q = 100;

    printf("%d %d %d %d\n", A[0], A[1], A[2], A[3]);
}
```

What output would be printed out at the terminal after compiling and executing this program?

- A. 1 2 3 4
- B. 1 2 3 100**
- C. 1 2 100 4
- D. 1 100 3 4
- E. None of the above

Answer:

q was initially pointing at **p[0]**. After the "***q++**", it points to **p[1]** (while modifying **p[0]** to point to **A[2]**) which contains the address of **A[3]**. The double dereferencing therefore changed what **p[1]** points to which is namely, **A[3]**. So the answer is option B.

Question 12

What is the hexadecimal encoding for the following instruction:

addi \$t1, \$sp, -120

- A. 0x23A90088
- B. 0x23A9FF88
- C. 0x239AFF88
- D. 0x293A0088
- E. None of the above

Answer:

This is a giveaway question. I expect everyone to get it right. The correct answer is B.

Question 13

Give the hexadecimal encoding for the following branch instruction:

bne \$a0, \$t8, EXIT

Assume that this instruction is at PC = 0x401C and EXIT is at PC = 0x35F0.

- A. 0x140035F0
- B. 0x1400401C
- C. 0x1498FD74
- D. 0x1498FD75
- E. None of the above

Answer:

This should also be easy. $PC+4 = 0x4020$. So the constant for the bne is $(0x35F0 - 0x4020)/4 = 0xFD74$ (16 bits two's complement). Hence, the answer is C.

Question 14

As a standard practice, we use an `ori` instruction to set a register with the 16-bit immediate zero extended to 32 bits, after a `lui` instruction has loaded the upper 16 bits. Which of the following MIPS instruction can also be used instead of the `ori` instruction to do the same thing?

- A. `addi`
- B. `andi`
- C. `slti`
- D. `nor`
- E. **None of the above**

Answer:

The answer is E.

We cannot use `addi` because the immediate will be sign extended, which may result in '1's in the upper 16 bits and this will affect the upper bits.

We cannot use `andi` because the lower 16 bits after `lui` is 0. And `and`'ing anything to it will still be zero.

We cannot use `slti` because the result is a comparison true (1) or false (0).

We cannot use `nor` because there is no immediate. We will need additional operations and that will complicate things.

Question 15

Consider the following MIPS instruction given in hexadecimals:

0x08012348

Assuming that the PC of this instruction is **0x2FFFFFFC**, what would be the PC after the execution of this instruction?

- A. **0x20012348**
- B. **0x2F048D20**
- C. **0x30048D20**
- D. **0x3001234C**
- E. None of the above

Answer:

PC+4 = 0x30000000. The address bits from the instruction are 0x012348. We multiply this by 4 to get 0x48D20. Appending this to the upper 4 bits of the PC yields the resultant PC of 0x30048D20, i.e. option C.

Question 16

According to the MIPS reference data sheet, the last occupied location on the top of the stack is pointed to by `$sp` and is word aligned. Which of the following would implement a pseudo stack push instruction that pushes the content of `$x` onto the stack?

- A. `sw $x, -4($sp)`
- B. `addi $sp, $sp, -4`
`sw $x, 0($sp)`
- C. `sw $x, -4($sp)`
`addi $sp, $sp, 4`
- D. `addi $sp, $sp, -4`
`lw $x, 0($sp)`
- E. None of the above

Answer:

The stack pointer first has to be decremented (since the stack grows towards the lower addresses) to yield an empty word before a `sw` is used to store it. So the answer is B.

For **Questions 17 and 18**, we will assume the parameters used in the standard MIPS encoding as shown in the MIPS Reference Sheet. In particular, we will assume that there are three instruction types: R-type, I-type, and J-type instructions. We will assume that for R-type, there are two subtypes – R-type integer and R-type floating point instructions that have the opcode of `0x00` and `0x11`, respectively.

Question 17

Given the above setup, how many distinct R-type integer instructions can be encoded?

- A. 32
- B. 64
- C. 128
- D. 256
- E. None of the above

Answer:

This is a giveaway question. For each R-type instruction opcode, we have 6-bits of **func** code that yields 64 instructions. So the answer is B.

Question 18

Suppose now we give up on having the shift instructions being of R-type and instead make them I-type, thus freeing up the bits used to encode the shift amounts. Further, suppose we use these free slots for integer instructions while keeping the R-type floating point operations unchanged. Now how many distinct R-type integer instructions can we encode?

- A. 256
- B. 1024
- C. 2048**
- D. 4096
- E. None of the above

Answer:

This would simply extend the **func** code to 11 bits and that will yield 2048 instructions. So the answer is C.

Question 19

Consider a 16-bit fixed length instruction set. Suppose there are three types of instructions:

- Type A: 3 operands, 3 bits each.
- Type B: 2 operands, 3 bits each.
- Type C: 1 3-bit long operand
- There must be at least one distinct instruction of each type.

What is the maximum number of instructions that can be formed using this specification?

- A. 3062
- B. 3584
- C. 8076
- D. 8122**
- E. None of the above

Answer:

Type A instructions require 9 bits for operands. Type B requires 6 bits for operands while Type C only needs 3. So for Type A, B and C instructions, we can have 7, 10 and 13 bits available as opcode. To maximize the number of instructions, we should have just 1 Type A, 1 Type B, and give all the opcode bits to Type C. We can have 3-tier opcode field with 7 bits for the first opcode, 3 bits for the next and another 3 bit for the third level. We can reserve 0000000_2 to be the opcode for the only Type A instruction. Then we can use 0000001000_2 for one Type B instruction. Then the second level opcode of 0000001001_2 (inclusive) onwards can be used for Type C. From 0000001001_2 to 111111111_2 there are 1014 slots. Each of these will allow for another 8 slots (000_2 - 111_2) at the third level. This yields $1015 \times 8 = 8120$. So the maximum would be $8120 + 1 \text{ Type A} + 1 \text{ Type B} = 8122$, option D.

Question 20

What is the minimum number of instructions that can be formed using the exact specification as in Question 19, and all opcodes are used?

- A. 198**
- B. 212
- C. 231
- D. 4867
- E. None of the above

Answer:

To minimize, we should give one first level opcode to Type C, one first level opcode to Type B and all others to Type A.

For Type C: since there is only one slot, let's say 0000000_2 at the first 7-bit level, and it has another 6 more bits (at the second and third level) which, if all opcodes are used, will yield 64 Type C instructions.

For Type B: let's give it say 0000001_2 as the first level 7-bit opcode. It has 3 more bits to specify 8 Type B instructions.

This leaves us with 0000010_2 to 1111111_2 to be used for Type A instructions. That would be 126 Type A instructions.

So number of Type A = 126, number of Type B = 8, number of Type C = 64. Total is 198 – assuming all opcodes are used. The answer is A.

Question 21

In the MIPS datapath taught in class, which of the following statements is false?

- A. The datapath to RR2 consists of 5-bits as you have 32 registers in total.
- B. RD2 is used by all instructions.**
- C. The ALU is a combinational circuit.
- D. In the 1-bit ALU, C_{in} is only used for the + operation.
- E. **\$zero** is a register that always returns the value zero and has no effect when it is written to.

Answer:

RD2 is not used by the `lw/sw` instructions.

Question 22

In the MIPS datapath taught in class, let us assume that `ALUoutput` is the output from the ALU. In the following, we check the value of `MemToReg` and then accordingly write to the register file.

`WriteData = MemToReg ? Memory[ALUoutput] : ALUoutput`

If we change the above datapath to the following:

`WriteData = ALUoutput`

which one of the following instructions would no longer work?

- A. `sw $s1, 2($s3)`
- B. `addi $s1, $s3, -50`
- C. `lw $s1, 2($s3)`**
- D. `j 0x12345678`
- E. None of the above

Answer:

The `lw` instruction needs the value from memory and can't just do with the `ALUoutput`.

Question 23

In the MIPS datapath taught in class, which of the following statements is true?

- A. The PC is a special register that is 36-bits long.
- B. **`ALUcontrol`** has 6-bits as its input and 4-bits as its output.
- C. **`ALUop`** is responsible for controlling the control signal **`RegDst`**.
- D. The PC is updated on the rising edge of the next clock cycle.**
- E. The PC is updated on the falling edge of the next clock cycle.

Answer:

From the definition of the clock and when the update to PC happens during the cycle.

Question 24

In the MIPS datapath taught in class, why are the inputs to the **MemToReg** multiplexer reversed?

- A. This is so that there is some variety in the datapath.
- B. The **MemToReg** multiplexer is the only one that deals with memory.
- C. **This is so that the wires do not cross over each other in the diagram.**
- D. It is because for the **sw** instruction, the source and destination registers are swapped
- E. None of the above.

Answer:

This is mainly to do with the diagram and to keep it clean.

Question 25

In the MIPS datapath taught in class, which of the following statements is false?

- A. MIPS has a total of 32 registers.
- B. MIPS opcodes are all 6 bits long.
- C. The **funct** field is sometimes used to set the **ALUcontrol** signal.
- D. The **ALUop** signal for the **beq** instruction is **01**.
- E. **None of the above.**

Answer:

All of the provided statements from A – D are true.

Part B: Short questions (3 questions, 5 marks each)

You must answer in the space provided in the Answer Sheets. Any writing outside the answer box will be disregarded.

Question 26

Suppose we do not have a real MIPS lui instruction. Instead, it is a pseudo-instruction of the form "**lui \$x, <16-bit const>**", where "**\$x**" is any of the valid registers (the assembler obtains the actual number) and the constant is 16-bit. We need to implement it using (the remaining) real MIPS instructions. The assembler uses a text rewriting process not unlike C macros. What you need to do is write a text template. Use "**\$x**" and "**<16-bit const>**" to represent the target register and the 16-bit constant in the original **lui** that the assembler will use it to instantiate an instance from your template and replace the line where lui is in the code with the instance. Show what your template looks like. Don't worry about style, the solution is to test the concept – though you have to be clear in your description, and your assumptions has to be realistic. You do have to be careful that your code must work in all code circumstances and not compromise values in the registers.

Answer:

// ori in the upper 16 bit constant but as a (low) 16 bit constant. Assembler does this.

ori \$x, \$zero, <16-bit const>

sll \$x, \$x, 16 // shift it to the upper 16 bits

Question 27

Suppose we want to design a new 32-bit fixed instruction set that is inspired by the MIPS encoding scheme. There are three types of instructions, i.e., R-type, I-type, and J-type. Now suppose the number of general purpose registers is increased to 64. Assuming that

- We do shifts using I-type instructions, thus doing away with the **shamt** field;
- The opcode field must be present but need not be 6 bits long.
- R-type instructions still has a **func** field whose length need not be 6;
- The immediate field is still 16-bit two's complement;
- There is at least ten R-type instruction;
- There is at least ten I-type instruction;
- We only need exactly two J-type instructions. The encoding is the same as in MIPS but the number of bits to be taken from the upper part of PC need not be 4. The address field should be as long as possible, requiring as few bits from the upper part of the PC as possible.

Design an expanding opcode scheme that **maximizes** the total number of instructions. Describe your design and fill in the numbers that are the result of your design in the respective boxes given.

Answer:

We will need 6 bits for each register field. We can then work out the constraint for the opcode field using the I-type instruction: it will need 16 bits for the immediate, 6 bits for rs and 6 bit for rt, leaving us 4 bits for the opcode. If **opcode** is 4 bits, and the three registers of rs, rt and rd in the R-type instruction will take up 18 bits, and since we no longer have the **shamt** field, we will therefore have 10 bits for the **func** field. So, if we keep 10 slots of the opcode for the "at least 10" I-type instructions, 2 slots for the J-type instruction, then we will have 4 slots for R-type instructions. Since each slot in the opcode can yield 2^{10} func codes, we have $4 \times 2^{10} = 4096$ R-type instructions. So:

Number of R-type instructions in your design:

4096

Number of I-type instructions in your design:

10

Maximum total number of instructions:

$4106 + 2 = 4108$

Question 28

Consider the MIPS datapath covered in class. For the MIPS instruction encoded as **0x10000103** fill in the corresponding elements in the boxes on the Answer Sheets. Use the notation \$R to represent register number R, [\$R] to represent the content of register number R and Mem(X) to represent the memory data at address X. Assume the PC value is 0x491 at the start.

Answer:

RR1:	\$0
WR:	\$0
WD:	0 or random value
Operand1 of ALU:	[\$0] or 0
New PC Value:	0x8A1

0x10000103 gives us the instruction **beq \$0, \$0, 0x103**

Faster is to do it in binary. Instruction is: 0001 0000 0000 0000 0000 0001 0000 0011. The rest can be filled in based on this.

The branch is taken, so new PC = PC + 4 + Immediate x 4 → 0x491 + 4 + 0x103 x 4 = 0x8A1

== END OF PAPER ==